

Tour Planner

Projektprotokoll – SWEN2

ASP.NET Core (C#) Backend · Angular Frontend · PostgreSQL

Projekt	Tour Planner – SWEN2
Autoren	Stefan Vukmirovic & Abdelrahman Salama
Datum	03.07.2026
Backend	ASP.NET Core (C#) – Layer-Architektur (API / BL / DAL / Models)
Frontend	Angular 20 – Standalone Components, MVVM
Datenbank	PostgreSQL via Entity Framework Core (Code-First)
Externe APIs	OpenRouteService.org (Routing) + Leaflet (Karte)
Tests	36 NUnit Unit Tests (100% bestanden)
Repository	github.com/if24b267/TourPlanner_SWEN2
Institution	FH Technikum Wien 2026

Inhaltsverzeichnis

1	Einleitung
2	Technische Schritte und Entscheidungen
2.1	Konfiguration & Logging
2.2	Datenhaltung mit PostgreSQL
2.3	OpenRouteService-Integration
2.4	Authentifizierung (Self-Register / Login)
2.5	Full-Text-Search inkl. berechneter Werte
2.6	Bild-Speicherung auf dem Filesystem
2.7	Import / Export
2.8	Mandatory Unique Feature: Achievement-System
2.9	Frontend-Integration
2.10	Bugfixing- und Code-Qualitäts-Runde
2.11	Bibliotheks-Entscheidungen & Lessons Learned
3	Anwendungsfälle (Use-Case-Diagramm)
4	UI-Design und Wireframes
5	Architektur
6	Unit Tests
7	Zeittracking
8	Anmerkung zur Versionskontrolle

1. Einleitung

Dieses Dokument beschreibt die technischen Entscheidungen, die Architektur und den Entwicklungsprozess der Tour-Planner-Anwendung im Rahmen der Lehrveranstaltung SWEN2. Die Anwendung ermöglicht es Nutzer:innen, eigene Touren (Rad-, Wander-, Lauf- oder Urlaubstouren) anzulegen, Tour-Logs zu erfassen und Statistiken über die eigenen Aktivitäten einzusehen.

Komponente	Technologie
Backend	ASP.NET Core (C#) – Layer-Architektur (Presentation / Business Logic / Data Access)
Frontend	Angular – Standalone Components
Datenbank	PostgreSQL via Entity Framework Core
Routing	OpenRouteService.org
Karte	Leaflet

2. Technische Schritte und Entscheidungen

Die Entwicklung erfolgte iterativ in klar abgegrenzten Schritten. Nachfolgend werden die wichtigsten Entscheidungen, aufgetretenen Probleme und deren Lösungen dokumentiert.

2.1 Konfiguration & Logging

Die Anwendungskonfiguration (Datenbank-Connection-String, Speicherpfad für Bilder, API-Keys, JWT-Secret) wird bewusst getrennt vom Quellcode gehalten: allgemeine, unkritische Defaults liegen in `appsettings.json` (wird versioniert), während lokale/geheime Werte in `appsettings.Development.json` liegen, die über `.gitignore` explizit von der Versionskontrolle ausgeschlossen ist. Für strukturiertes Logging wurde **Serilog** eingesetzt (Konsolen- und Datei-Sink mit täglicher Rotation), als funktionales Äquivalent zu `log4net/log4j` im .NET-Ökosystem.

2.2 Datenhaltung mit PostgreSQL

Ursprünglich war im Grundgerüst SQLite als Platzhalter-Provider eingetragen. Da die Angabe explizit PostgreSQL fordert, wurde auf `Npgsql.EntityFrameworkCore.PostgreSQL` umgestellt. Zur Isolation von anderen, bereits lokal laufenden Datenbank-Containern (aus Vorprojekten) wurde ein eigener, projektspezifischer PostgreSQL-Container per `docker-compose` auf einem alternativen Port betrieben. Die Tabellenstruktur wird vollständig über EF-Core-Migrationen verwaltet (Code-First-Ansatz).

2.3 OpenRouteService-Integration

Route (Distanz, Zeit, Geometrie) wird über die REST-API von `openrouteservice.org` berechnet: zunächst Geocoding von Start-/Zieladresse, danach Routing-Anfrage passend zum gewählten Transportmittel (Mapping z. B. "Hike" → `foot-hiking`, "Bike" → `cycling-regular`). Da die Freischaltung eines produktiven API-Keys durch den Anbieter (HeiGIT) zeitlich verzögert war, wurde die Integration bewusst mit einem Platzhalter-Key entwickelt und mit einem **Fallback-Mechanismus** abgesichert: schlägt der externe Aufruf fehl (z. B. 401 durch ungültigen Key), fängt der `TourService` die Exception ab, protokolliert eine Warnung und weicht auf grob geschätzte Werte aus, damit die Anwendung durchgehend funktionsfähig bleibt. Sobald ein gültiger Key vorliegt, ist keine Codeänderung nötig – nur der Konfigurationswert muss ersetzt werden.

2.4 Authentifizierung (Self-Register / Login)

Für die Nutzer-Authentifizierung wurde **JWT (JSON Web Tokens)** gewählt, da Backend und Frontend als getrennte Zwei-Schichten-Architektur über REST kommunizieren – ein zustandsloses Token-Verfahren passt hier besser als serverseitige Cookie-Sessions, die zusätzliches CORS-/SameSite-Handling nötig gemacht hätten. Passwörter werden mit ASP.NET Core Identity PasswordHasher gehasht gespeichert, niemals im Klartext. Jede Tour und jeder Tour-Log ist über eine `UserId` referenzierbar; sämtliche Endpunkte filtern serverseitig nach dem aus dem JWT extrahierten Benutzer, sodass kein Zugriff auf fremde Daten möglich ist ("No sharing of anything").

2.5 Full-Text-Search inkl. berechneter Werte

Die Suche berücksichtigt sowohl die textuellen Felder (Name, Beschreibung, Start/Ziel, Log-Kommentare) als auch die abgeleiteten Werte **Popularity** und **Child-Friendliness**. Da diese Werte als `[NotMapped]`-Properties nur in C# berechnet werden (nicht in der Datenbank existieren), erfolgt die Filterung nach dem Laden

in-memory. Suchbegriffe können sowohl klassische Textsuche als auch Schlagworte ("beliebt", "neu", "kinderfreundlich") sowie numerische Werte auslösen, die direkt gegen Popularity bzw. Child-Friendliness geprüft werden.

2.6 Bild-Speicherung auf dem Filesystem

Bilder werden nicht in der Datenbank, sondern als Binärdateien auf dem Filesystem abgelegt (Pfad konfigurierbar über StorageOptions). In der Datenbank wird nur der relative Dateiname referenziert. Ein eigener **ImageStorageService** kapselt Speichern, Abrufen und Löschen; erlaubte Dateitypen sind auf gängige Bildformate beschränkt.

2.7 Import / Export

Der Export erzeugt eine JSON-Datei mit sämtlichen Touren und zugehörigen Tour-Logs des angemeldeten Benutzers. Bewusst wurden die ursprünglichen GUIDs beibehalten, damit ein Re-Import nach vollständigem Löschen aller Daten möglich ist, ohne Referenzen zu verlieren (Roundtrip-Fähigkeit). Beim Import werden bereits existierende Touren (gleiche Id) übersprungen, um Duplikate bei mehrfachem Import zu vermeiden.

2.8 Mandatory Unique Feature: Achievement-System

Als eigenständiges, über die Pflichtenforderungen hinausgehendes Feature wurde ein **Achievement-/Gamification-System** umgesetzt. Nach jeder neuen Tour bzw. jedem neuen Tour-Log werden acht unterschiedliche Kriterien automatisch ausgewertet (u. a. Gesamtdistanz, Einzel-Zeitdauer, Schwierigkeitsgrad, Durchschnittsbewertung, Anzahl Touren, zeitliche Häufung von Logs). Freigeschaltete Achievements werden persistiert und im Frontend als Badge-Übersicht angezeigt. Das Feature ist bewusst nicht als reine Erweiterung bestehender CRUD-Funktionalität konzipiert, sondern als eigenständige Domäne mit eigener Business-Logik, eigenem Datenmodell (UserAchievement) und eigenem Endpunkt.

2.9 Frontend-Integration

Das Frontend wurde um folgende Bausteine erweitert:

- Login-/Registrierungs-Komponente inkl. HTTP-Interceptor, der den JWT automatisch an jeden Request anhängt
- Eine echte Leaflet-Kartenkomponente (ersetzt den ursprünglichen Platzhalter) mit sauberem Fallback-Overlay, solange keine Routen-Geometrie vorliegt
- Bild-Upload direkt in der Tour-Detailansicht
- Ein Achievements-Panel
- Buttons für Export/Import

2.10 Bugfixing- und Code-Qualitäts-Runde

Nach der ersten funktionsfähigen Version wurde das Projekt im Team einer gezielten Fehlersuche unterzogen (Branch `fix/tour-log-editing-and-backend-fixes`). Die dabei gefundenen Probleme, deren Ursache und die gewählte Lösung werden hier dokumentiert, da sie Teil des geforderten Protokolls über Entwicklungsentscheidungen und aufgetretene Fehlschläge sind.

Funktionale Bugfixes:

Problem	Lösung
---------	--------

Tour-Bearbeitung persistierte nicht	Der "Save"-Button im Edit-Formular rief nie den Update-Endpunkt auf. Lösung: eigene Arbeitskopie (editTour) eingeführt, onSaveEdit() ruft jetzt tatsächlich tourService.update() mit Validierung auf.
Tour-Log-Bearbeitung fehlte in der UI	Obwohl das Backend sie längst unterstützte: TourLogForm erhielt einen optionalen existingLog-Input inkl. Edit-Button pro Log-Eintrag.
Tour-Wechsel während offener Bearbeitung	Konnte die falsche Tour überschreiben: ngOnChanges setzt den Bearbeitungszustand jetzt beim Wechsel der gebundenen Tour zurück.
Sidebar zeigte nach dem Speichern veraltete Werte	Da die Objektreferenz ersetzt statt gemergt wurde: Fix per Object.assign(); dafür musste TourService.UpdateAsync im Backend um .Include(t => t.TourLogs) ergänzt werden, da Popularity/Child-Friendliness sonst fälschlich auf 0 zurückgefallen wären.
IDOR-Sicherheitslücke bei Tour-Logs	Update/Delete prüften nur, ob die Tour dem eingeloggten User gehört, nicht ob der übergebene Log tatsächlich zu dieser Tour gehört. Mit einer fremden Log-ID und einer eigenen Tour-ID ließ sich so ein fremder Log ändern oder löschen. Fix: Zugehörigkeitsprüfung per Tour-Id direkt im Query, gebündelt in GetOwnedLogAsync().
Transport-Typ-Mismatch	Frontend bot "Foot"/"Train" an, das Backend/ORS-Mapping kannte nur Car/Bike/Hike/Running/Vacation – bei Nichtübereinstimmung fiel die Route lautlos auf driving-car zurück. Fix: gemeinsame TRANSPORT_TYPES-Konstante, deckungsgleich mit dem Backend-Mapping.
Stale Routendaten nach fehlgeschlagenem Geocoding	Bei einem Tour-Update griff der Fallback nur, wenn die Distanz noch 0 war. Fix: Fallback greift jetzt bei jedem Fehlschlag neu.
Fehlende DI-Registrierung für ImageStorageService	Jeder Aufruf des Bild-Upload-Endpunkts crashte serverseitig mit einer InvalidOperationException. Dies war die Ursache für den zuvor beobachteten fehlgeschlagenen Bild-Upload-Test (siehe Kapitel 9). Fix: Registrierung in Program.cs ergänzt.
Abgelaufene Session (JWT nach 120 Min.)	Wirkte wie ein eingefrorenes UI, da 401-Antworten nicht erkannt wurden: der Interceptor meldet 401 jetzt an einen sessionExpired-Zustand, der automatisch ausgelogged und eine klare Meldung anzeigt.

Architektur / Code-Qualität:

- Eigene Exception-Hierarchie (BusinessException, EntityNotFoundException, UnsupportedFileTypeException, RouteLookupException) statt generischer throw new Exception(...) aus der Business-Schicht; Controller fangen jetzt gezielt diese Typen ab.
- Globale Exception-Handling-Middleware (UseExceptionHandler) protokolliert unbehandelte Exceptions über Serilog und liefert eine saubere JSON-500-Antwort statt eines nackten Fehlers.
- Der Catch-Block in PopulateRouteDataAsync fängt nur noch erwartete Fehlerarten (RouteLookupException, HttpRequestException, JsonException, TaskCanceledException), statt pauschal jede Exception als "ORS nicht erreichbar" zu maskieren.
- Responsive Design ergänzt: gemeinsame SCSS-Breakpoints (_breakpoints.scss) und Media-Queries, sodass sich Sidebar/Detail-Layout, Formulare und das Achievements-Panel auf mobilen Bildschirmen sinnvoll anpassen – vorher gab es im gesamten Frontend keine einzige Media-Query.
- Aufräumarbeiten: übrig gebliebener NUnit-Scaffold-Test (UnitTest1.cs, Assert.Pass()) sowie ungenutzte, leere app.config.ts/app.routes.ts entfernt.

Die vollständige Liste der geänderten Dateien ist Teil der Commit-Historie auf dem genannten Branch und wird hier gemäß Kapitel 8 nicht dupliziert.

2.11 Bibliotheks-Entscheidungen & Lessons Learned

Bibliotheks-/Framework-Entscheidungen:

Bibliothek	Begründung
Serilog	Funktional gleichwertig zu log4net (strukturiertes Logging, Datei-Rotation), aber idiomatischer für ASP.NET Core (native Integration, keine XML-Konfiguration nötig).
Npgsql.EntityFrameworkCore.PostgreSQL	Von der Angabe für PostgreSQL vorgeschrieben; ersetzte den ursprünglich im Grundgerüst vorhandenen SQLite-Provider.
JWT (System.IdentityModel.Tokens.Jwt)	Passt besser zu einer echten Zwei-Schichten-Architektur mit getrenntem Frontend/Backend über REST als cookie-basierte Sessions (siehe Kapitel 2.4).
Leaflet	Von der Angabe vorgeschrieben, außerdem lizenzkostenfrei nutzbar mit OpenStreetMap-Kacheln, kein API-Key für die reine Kartenanzeige nötig.
NUnit + Moq + EF InMemory	NUnit von der Angabe vorgeschrieben; Moq zum Isolieren externer Abhängigkeiten; InMemory-Provider erlaubt DB-abhängige Tests ohne echte PostgreSQL-Instanz.

Lessons Learned:

- Frameworks/Editoren verlassen sich stark auf exakte Datei- und Ordnernamen (z. B. docker-compose.yml statt docker.compose.yml, oder .gitignore-Pfade, die exakt der Ordnertiefe entsprechen müssen) – kleine Tippfehler haben wiederholt zu großem Debugging-Aufwand geführt.
- Bei mehreren EF-Core-Projekten (API/BL/DAL) müssen NuGet-Paketversionen projektübergreifend konsistent gehalten werden; ein Versions-Mismatch führt zu schwer lesbaren Compiler-Fehlern in generierten Migrations-Dateien.
- Externe APIs (hier: OpenRouteService/HeiGIT) sollten von Beginn an hinter einem Interface mit klar definiertem Fallback-Verhalten entwickelt werden, statt die Kernfunktionalität von der rechtzeitigen Freischaltung eines API-Keys abhängig zu machen.
- Sicherheitslücken wie IDOR entstehen leicht, wenn Berechtigungsprüfungen nur auf oberster Ebene stattfinden, aber nicht auf verschachtelten Ressourcen. Eine explizite, wiederverwendbare Prüfmethode pro Ressourcentyp (hier: GetOwnedLogAsync) beugt dem systematisch vor.
- Swagger benötigt für Datei-Uploads (IFormFile) und für JWT-Authorization explizite Zusatzkonfiguration (Consumes-Attribut bzw. SecurityDefinition) – ohne diese bleiben Funktionen im UI unsichtbar oder nicht testbar.

3. Anwendungsfälle (Use-Case-Diagramm)

Das folgende UML-Use-Case-Diagramm zeigt die vom Benutzer direkt auslösbaren Anwendungsfälle sowie die drei automatisch vom System ausgeführten Folgeaktionen (Routenberechnung, Berechnung abgeleiteter Werte, Achievement-Auswertung), die jeweils per «include»-Beziehung an die entsprechenden Benutzeraktionen gekoppelt sind.

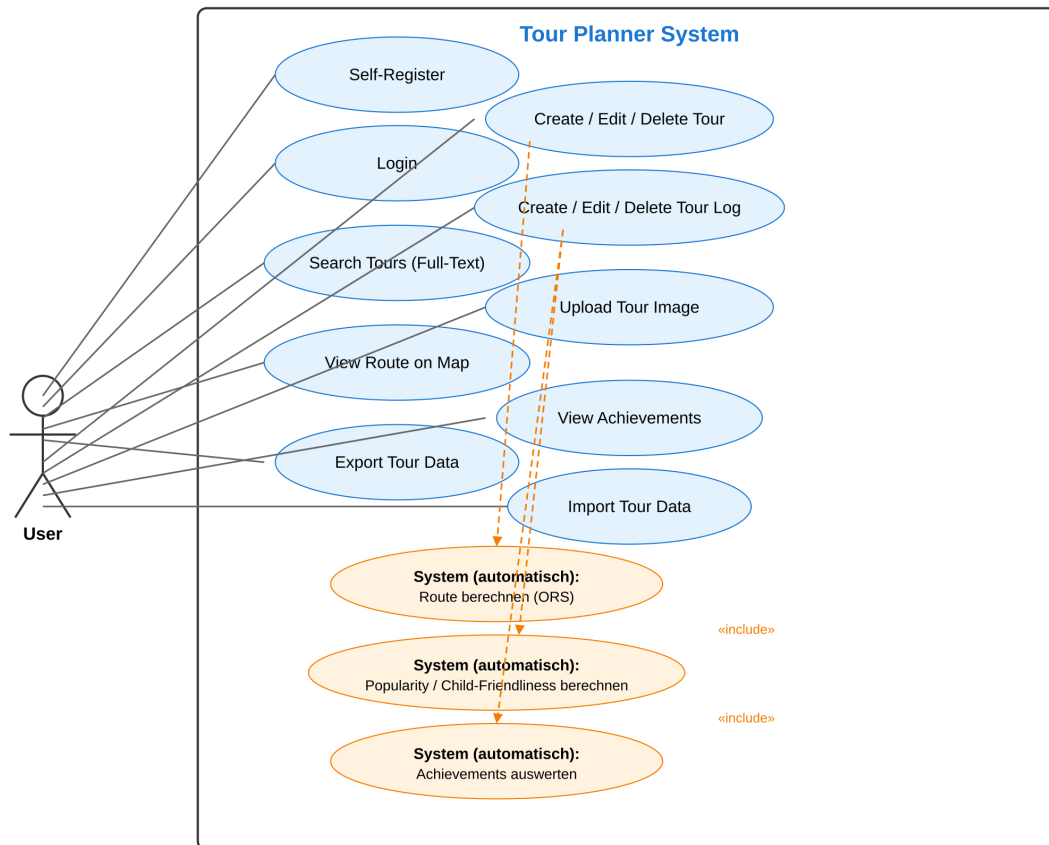


Abbildung 1: UML Use-Case-Diagramm – Tour Planner

Abbildung 1: UML Use-Case-Diagramm – Tour Planner

4. UI-Design und Wireframes

Das UI-Design folgt einer klassischen **Master-Detail-Ansicht**: eine Liste aller Touren links, Detailansicht (inkl. Karte, Bild und Logs) rechts. Diese Struktur wurde bewusst gewählt, weil sie den Kernanwendungsfall (Tour auswählen → Details/Logs verwalten) mit minimaler Klickzahl unterstützt. Nachfolgend die Low-Fidelity-Wireframes der drei zentralen Bildschirme.

The wireframe shows a central box titled "[Login / Register]". Inside, there are two input fields labeled "Username" and "Passwort". Below these is a button labeled "[Einloggen]". At the bottom, there is a link that says "Noch kein Konto? Registrieren (Link)".

Abbildung 2: Wireframe – Login / Registrierung

The wireframe is titled "Tour Planner". It features a top navigation bar with "[Achievements]" and "[Logout]". Below this is a search bar labeled "Search tours..." with buttons for "[Search]", "[+ New]", "[Export]", and "[Import]". The main content area is split into two columns. The left column contains a list of tour cards, each with "Tour Name", "From -> To", and "[Badge] Popularity: n". The right column shows a detailed view for a selected tour, including a "Tour Name" header with "[Edit]" and "[Delete]" buttons, a "Description / From-To / Transport / Distance / Time / Popularity / Child-Friendliness" section, a map placeholder labeled "[Leaflet Map: Route from -> to]", a "[Tour Image / Upload]" section, and a "Tour Logs (n)" section with a "[+ Add Log]" button. The logs section includes a table header: "Date | Rating | Comment | Difficulty | Distance | Time | [Delete]".

Abbildung 3: Wireframe – Tour-Übersicht mit Detailansicht, Karte, Bild und Logs

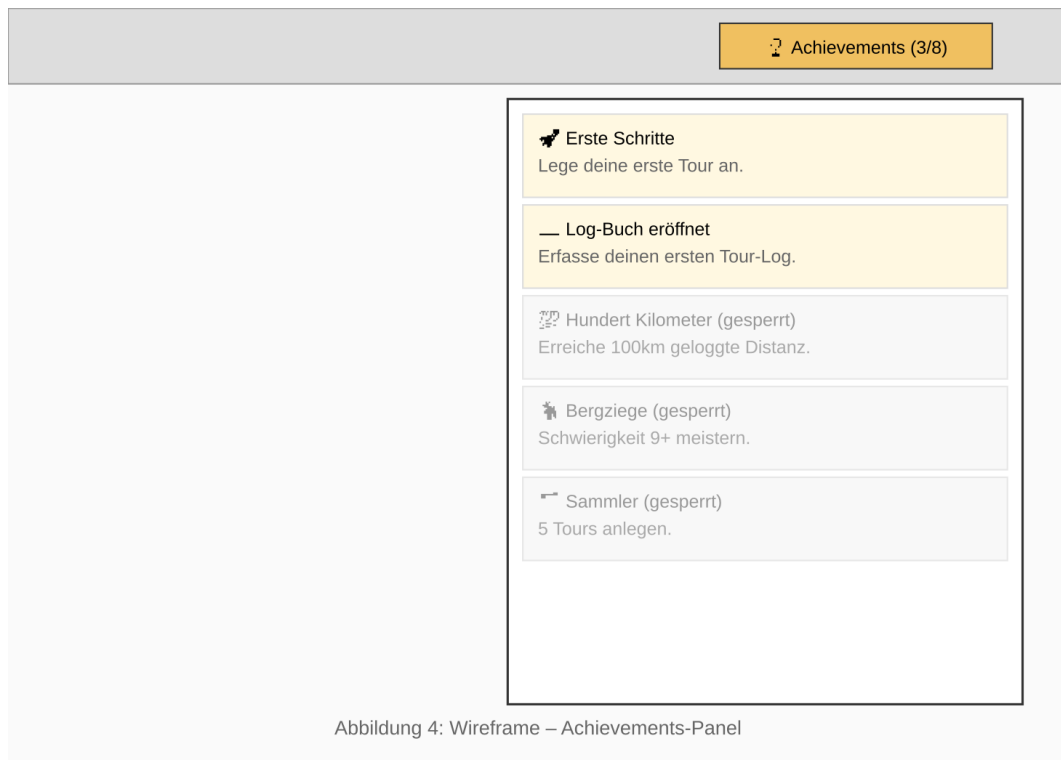


Abbildung 4: Wireframe – Achievements-Panel

Abbildung 4: Wireframe – Achievements-Panel

5. Architektur

5.1 Layer-Architektur

Die Anwendung ist in vier .NET-Projekte gegliedert, die eine klare Trennung von Verantwortlichkeiten erzwingen:

Projekt	Layer	Verantwortlichkeit
TourPlanner.API	Presentation Layer	Controller, Auth-Middleware, Swagger, DI-Konfiguration
TourPlanner.BL	Business Logic Layer	Services (TourService, AuthService, AchievementService, ImportExportService, OpenRouteService, ImageStorageService), Konfigurationsklassen, Achievement-Katalog
TourPlanner.DAL	Data Access Layer	EF-Core DbContext, Migrationen
TourPlanner.Models	Shared Models	Gemeinsam genutzte Entitäten (Tour, TourLog, User, UserAchievement)

Die Kommunikation zwischen den Schichten erfolgt ausschließlich über Interfaces (z. B. ITourService, IRouteService), die per Dependency Injection in den Konstruktoren bereitgestellt werden. Dies entkoppelt die Schichten und ist Grundlage für die Testbarkeit (siehe Kapitel 6).

5.2 MVVM im Frontend

Das Angular-Frontend folgt dem **MVVM-Pattern** (Model-View-ViewModel), wie von der Angabe explizit gefordert:

Schicht	Implementierung	Beschreibung
Model	TypeScript-Interfaces in models/ + Services	Datenzugriff über HTTP, Rohdaten bereitstellen
ViewModel	Standalone-Components (TourList, TourDetail, TourForm ...)	UI-Zustand, Service-Aufrufe, Datentransformation für View
View	HTML-Templates (tour-list.html ...)	Ausschließlich Angular-Databinding, keine Business-Logik im Template

Konkretes Beispiel: In tour-list.ts hält die Komponente die Property tours: Tour[] (ViewModel-Zustand) und die Methode onSearch(), die den TourService aufruft. Das Template tour-list.html bindet die Liste rein deklarativ per *ngFor, ohne selbst Logik auszuführen – Änderungen am ViewModel-Zustand aktualisieren die View automatisch über Angulars Change Detection.

5.3 Eingesetzte Design Patterns

Pattern	Anwendung
---------	-----------

Strategy Pattern	Die Achievement-Kriterien sind als austauschbare Funktionsobjekte (Func, bool>) im AchievementCatalog definiert. Jedes Kriterium kapselt eine eigene Berechnungsstrategie (Summenbildung, Durchschnittsbildung, Sliding-Window-Analyse etc.), ohne dass der aufrufende AchievementService deren konkrete Implementierung kennen muss. Neue Achievements lassen sich durch Hinzufügen eines weiteren Katalogeintrags ergänzen, ohne bestehenden Code zu verändern (Open/Closed-Prinzip).
Dependency Injection	Sämtliche Services werden über den integrierten DI-Container von ASP.NET Core registriert und injiziert (z. B. IRouteService, das je nach Konfiguration gegen eine Test-Implementierung (Moq) oder die echte OpenRouteService-Implementierung ausgetauscht werden kann).
Repository-ähnliches Muster über EF Core	Der TourDbContext kapselt den Datenbankzugriff; die Business-Logic-Services greifen nie direkt auf SQL zu, sondern ausschließlich über das DbContext-Abstraktionslevel.

5.4 Klassendiagramm

Das folgende Klassendiagramm zeigt die zentralen Entitäten (Models), die Interfaces der Business-Logic-Schicht sowie deren konkrete Implementierungen und den DbContext.

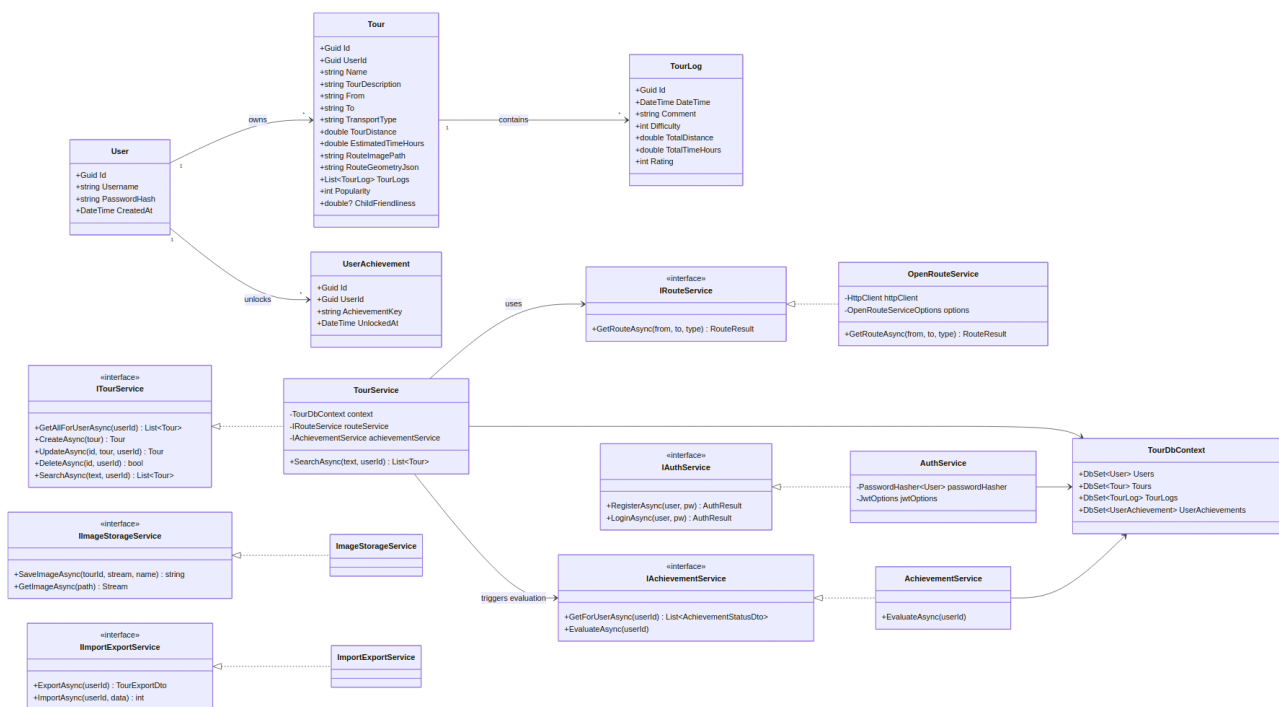


Abbildung 5: UML-Klassendiagramm der Business-Logic- und Modellschicht

5.5 Sequenzdiagramm: Full-Text-Search

Das folgende Sequenzdiagramm zeigt den Ablauf einer Volltextsuche vom Klick im Frontend bis zur gefilterten Antwort, inklusive der Berücksichtigung der berechneten Werte (Popularity, Child-Friendliness).

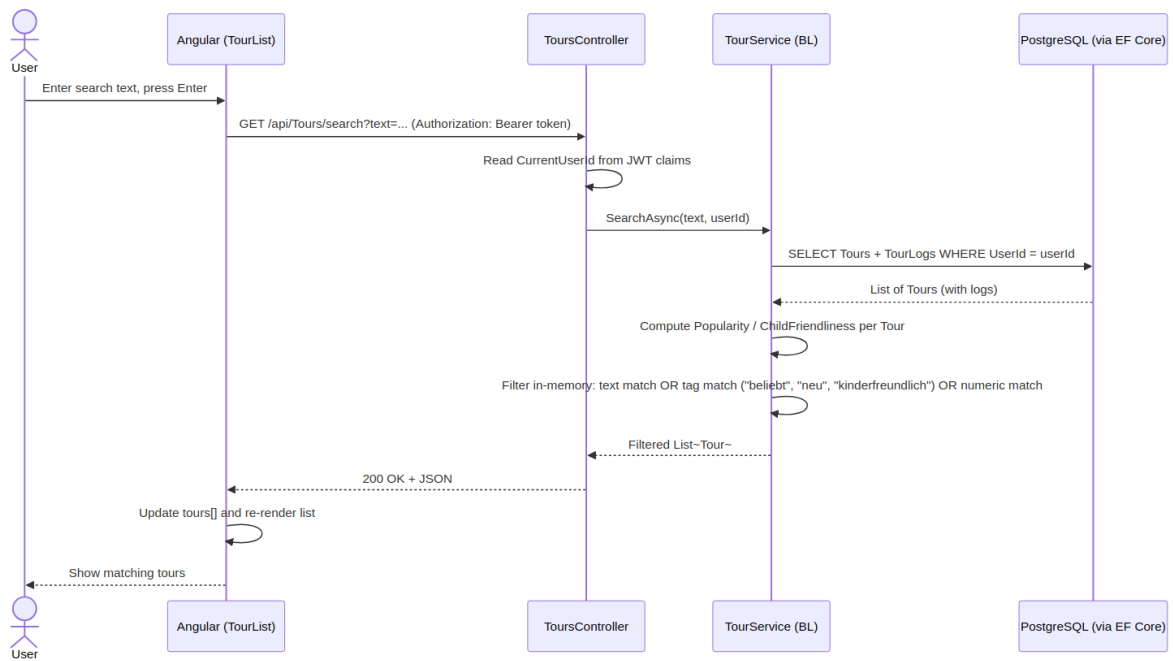


Abbildung 6: Sequenzdiagramm – Full-Text-Search

6. Unit Tests

Es wurden **36 automatisierte Unit Tests** mit NUnit umgesetzt (mehr als die geforderten 20), bewusst über sechs fachlich unterschiedliche Kategorien verteilt, damit die Tests unterschiedliche Arten von Logik und nicht nur Variationen desselben Testfalls abdecken. (Hinweis: dotnet test zeigte während der Entwicklung zwischenzeitlich 37 Tests, da ein ungenutzter NUnit-Scaffold-Test aus dem Projekt-Grundgerüst mitgezählt wurde; dieser wurde im Rahmen der Aufräumarbeiten in Kapitel 2.10 entfernt.)

Kategorie	Anzahl	Warum kritisch
Model-Tests (berechnete Properties)	3	Popularity/Child-Friendliness sind reine Berechnungen ohne DB-Zugriff; ein Fehler hier würde sich unbemerkt durch die gesamte Suche und Anzeige ziehen.
Achievement-Tests	16	Kernstück des Unique Features. Jedes der 8 Kriterien prüft eine andere Berechnungsart (Summe, Durchschnitt, Sliding-Window, einfacher Count) – Fehler wären für Nutzer:innen unsichtbar bis ein erwartetes Badge ausbleibt.
Auth-Tests	6	Sicherheitskritisch: falsches Passwortverhalten, doppelte Usernamen oder fehlerhafte Tokenausstellung wären direkte Sicherheitslücken.
Search-Tests	5	Deckt die geforderte Berücksichtigung der berechneten Werte in der Volltextsuche ab; leicht kaputt durch Refactoring der Popularity-Logik.
Route-Fallback-Tests	2	Stellt sicher, dass ein Ausfall des externen OpenRouteService (z. B. ungültiger Key) die Anwendung nicht zum Absturz bringt.
CRUD/Ownership-Tests	4	Verhindert, dass Benutzer:innen auf fremde Touren zugreifen, sie ändern oder löschen können – direkte Verletzung der Anforderung "No sharing of anything" wäre sonst ein stiller Bug.
Gesamt	36	36/36 bestanden (dotnet test)

Alle Tests laufen gegen eine EF-Core-InMemory-Datenbank (statt PostgreSQL) und mocken externe Abhängigkeiten (IRouteService, IAchievementService) mit Moq, wodurch sie schnell, deterministisch und unabhängig von einer laufenden Infrastruktur ausführbar sind. Testergebnis: **36/36 bestanden** (dotnet test).

7. Zeittracking

Die folgenden Stunden sind auf Basis des Projektverlaufs konsolidiert erfasst; kleinere Abweichungen sind möglich und können bei Bedarf noch händisch nachjustiert werden.

Phase	Vukmi (h)	Abdel (h)	Beschreibung
Setup & Konfiguration	4	2	Projektstruktur, Docker, Postgres, Logging
Backend – Auth	5	3	JWT, Register/Login, Ownership
Backend – OpenRouteService	4	3	Geocoding, Routing, Fallback
Backend – Bild-Storage	3	1	Upload/Abruf/Löschen von Bildern
Backend – Search	3	1	Volltextsuche inkl. berechneter Werte
Backend – Import/Export	2	1	JSON Export/Import, Roundtrip
Backend – Unique Feature	4	2	Achievement-System
Backend – Unit Tests	4	2	36 NUnit-Tests
Frontend – Auth-Integration	2	2	Login/Register-UI, Interceptor
Frontend – Leaflet-Karte	2	2	Ersetzung des Platzhalters
Frontend – Bild-Upload-UI	2	2	
Frontend – Achievements-UI	2	2	
Frontend – Import/Export-UI	2	1	
Bugfixing & Code-Qualität	1	8	Branch fix/tour-log-editing-and-backend-fixes (siehe Kapitel 2.10)
Live-Testing & QA	1	3	Bild-Upload, Achievements, Import/Export-Roundtrip
Recherche ORS (HeiGIT)	0	2	Account-Freischaltung
Code Reviews / Abstimmung	2	2	
Dokumentation	2	1	Protokoll, Diagramme, Wireframes
Gesamt	45	40	

8. Anmerkung zur Versionskontrolle

Die Git-Historie des Repositories ist Teil der Projektdokumentation und wird hier bewusst nicht dupliziert. Sie dokumentiert die schrittweise Entwicklung der einzelnen Features:

Config → Datenbank → OpenRouteService → Auth → Bild-Storage → Search → Import/Export →
Achievement-System → Unit Tests → Frontend-Integration → Bugfixing-Runde

Ressource	URL
Git-Repository	https://github.com/if24b267/TourPlanner_SWEN2.git

Autoren: Stefan Vukmirovic & Abdelrahman Salama — FH Technikum Wien, 2026